ASSIGNMENT REPORT 1: EVALUATING PERFORMANCE BOTTLENECKS: THREADS VS. PROCESSES

CENG 2034, OPERATING SYSTEMS

Neval Doğru
nevaldogru@posta.mu.edu.tr

Sunday 3rd May, 2026

Abstract

This report analyzes the performance differences between multi-threading and multi-processing in Python using I/O-bound and CPU-bound scenarios. By testing various workloads on a 10-core system, I identified the Efficiency Cross-over point where the parallelism of processes overcomes the Global Interpreter Lock (GIL) limitations and the inherent overhead of process management. The report specifically explores how address space isolation and communication costs impact total execution time.

1 Introduction

Concurrent programming in Python is primarily governed by the Global Interpreter Lock (GIL). As defined in technical literature, the GIL is a mutex that prevents multiple native threads from executing Python bytecodes simultaneously to ensure thread-safe memory management in CPython. While it allows for efficient execution in I/O-bound tasks where the CPU frequently waits, it creates a bottleneck for CPU-bound tasks by preventing true parallel execution across multiple cores.

A critical factor in this performance analysis is the nature of Address Spaces. Threads within the same process share a common address space, including the heap and data sections, which facilitates fast communication and lower context-switching costs. In contrast, processes possess isolated address spaces assigned by the operating system. According to Algomaster, this isolation provides stability but necessitates Inter-Process Communication (IPC) mechanisms for data sharing. The complexity of IPC and the higher cost of context switching—due to the OS needing to switch memory mappings constitute the Communication Tax. This tax is the primary reason multiprocessing often underperforms at lower workloads compared to threading.

2 Assignments

The implementation was developed in Python using the `threading` and `multiprocessing` modules. I utilized 10 workers to align with the physical core count of the system.

2.1 Assignment 1.1: The I/O Simulation

The I/O-bound task simulated 50 operations with a 0.1-second delay using `time.sleep(0.1)`. To maintain a consistent task count across threads, a `threading.Lock` was used to protect the shared counter. For the multiprocessing approach, a `multiprocessing.Value` object was utilized to manage the counter across isolated memory spaces.

2.2 Assignment 1.2: Finding the Break-even Point

I created a CPU-intensive function (`check_even_odd`) to identify the break-even point where multiprocessing becomes more efficient. The workload was incremented from 40,000,000 to 80,000,000. This experiment aimed to determine when the benefits of bypassing the GIL on 10 cores outweigh the Communication Tax associated with IPC and process initialization.

3 Results

The tests were performed on a system with the hardware specifications listed in Table 1.

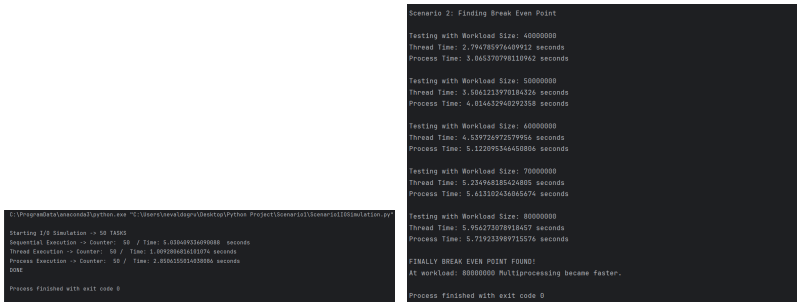
Component	**My Laptop Specifications**
CPU	12th Gen Intel(R) Core(TM) i7-1255U
Core Count	10 Cores (12 Threads)
RAM	16.0 GB
Operating System	Windows 11 Pro

Table 1: Hardware configuration used for testing.

Experimental results show that for workloads under 80,000,000, threading remained faster due to the shared address space advantage. However, at 80,000,000 operations, the process execution time (5.71s) became superior to threading (5.95s) as the GIL became the dominant bottleneck.

Workload Size	**Thread Time (s)**	**Process Time (s)**	**Winner**
40,000,000	2.794	3.065	Threading
50,000,000	3.506	4.014	Threading
60,000,000	4.539	5.122	Threading
70,000,000	5.234	5.613	Threading
80,000,000	**5.956**	**5.719**	**Multiprocessing**

Table 2: Performance comparison for Scenario 2.



(a) Scenario 1 Results (b) Scenario 2 Results

Figure 1: Terminal output comparisons showing threading's advantage in I/O and the break-even at 80M for CPU tasks.

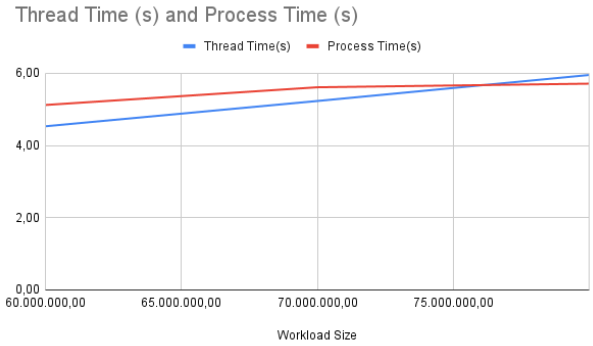


Figure 2: The efficiency cross-over graph where the process curve intersects and falls below the thread curve at 80M.

4 Conclusion

The study demonstrates that multi-threading is more effective for I/O-bound tasks due to its shared address space and lower context-switching overhead. For CPU-bound tasks, however, the GIL severely limits performance scaling. On a 10-core i7 processor, the threshold where the advantages of true parallelism surpass the Communication Tax was found to be 80,000,000 operations. Beyond this point, the OS level isolation and IPC costs are justified by the significant reduction in total execution time.

References

[1] CENG 2034 Operating Systems, Assignment 3: The Concurrency Threshold, Muğla Sıtkı Koçman University.

[2] Wikipedia contributors, "Global Interpreter Lock," Wikipedia, `https://en.wikipedia.org/wiki/Global_interpreter_lock` (Accessed May 3, 2026).

[3] Algomaster, "Processes vs Threads," https://algomaster.io/learn/concurrency-interview/processes-vs-threads (Accessed May 3, 2026).